

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО»

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ №5

«ПРОЦЕДУРЫ, ФУНКЦИИ, ТРИГГЕРЫ В POSTGRESQL»

по дисциплине «Проектирование и реализация баз данных»

Обучающийся Янин Егор Вячеславович

Факультет ФПИИ

Группа К3241

Направление подготовки 09.03.03 Прикладная информатика

Образовательная программа Мобильные и сетевые технологии 2024

Преподаватель Говорова Марина Михайловна

Санкт-Петербург
2025/2026

Цель работы

Овладеть практическими создания и использования процедур, функций и триггеров в базе данных PostgreSQL.

Практическое задание

1. Создать 3 процедуры для индивидуальной БД согласно варианту (часть 4 ЛР 2). Допустимо использование IN/INOUT параметров. Допустимо создать авторские процедуры. (3 балла)
2. Создать триггеры для индивидуальной БД согласно варианту:

Вариант 2.1. 7 триггеров по ИЗ.

Вариант 2.2. 5 оригинальных триггеров + модификация триггерной функции Части 2 практического занятия доп. баллы).

Выполнение

Процедуры:

1. Вывести сведения о заказах заданного официанта на заданную дату.

```
CREATE OR REPLACE PROCEDURE get_waiter_orders_proc(
    IN p_employee_id INT,
    IN p_date DATE,
    INOUT ref REFCURSOR
)
AS $$
BEGIN
    OPEN ref FOR
    SELECT
        o.id_order,
        o.order_code,
        o.order_time,
        o.order_cost
    FROM orders o
    JOIN employee_shift es
        ON es.id_employee_shift = o.id_employee_shift
    WHERE es.id_employee = p_employee_id
        AND o.order_time::DATE = p_date;
END;
$$ LANGUAGE plpgsql;

db_restaurant=# BEGIN;
BEGIN
db_restaurant==# CALL get_waiter_orders_proc(4, DATE '2026-03-20', 'orders_cursor');
ref
-----
orders_cursor
(1 строка)

db_restaurant==# FETCH ALL FROM orders_cursor;
 id_order | order_code | order_time | order_cost
-----+-----+-----+-----
         4 | cbd7d833e6 | 2026-03-20 12:06:25.572181 | 760,00 ?
(1 строка)

db_restaurant==# COMMIT;
COMMIT
```

2. Выполнить расчет стоимости заданного заказа.

```
CREATE OR REPLACE PROCEDURE calculate_order_cost(
    p_order_id INT
)
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE orders o
    SET order_cost = (
        SELECT SUM(dp.dish_price * dor.dish_number)
        FROM dish_order dor
```

```

        JOIN dish_price dp
        ON dp.id_dish = dor.id_dish
        WHERE dor.id_order = p_order_id
    )
    WHERE o.id_order = p_order_id;
END;
$$;

db_restaurant=# SELECT order_cost FROM orders WHERE id_order = 1;
 order_cost
-----
      0,00 ?
(1 строка)

db_restaurant=# CALL calculate_order_cost(1);
CALL
db_restaurant=# SELECT order_cost FROM orders WHERE id_order = 1;
 order_cost
-----
 2 460,00 ?
(1 строка)

```

3. Повышения оклада заданного сотрудника на 30 % при повышении его категории.

```

CREATE OR REPLACE PROCEDURE promote_employee(
    p_employee_id INT,
    p_new_category TEXT
)
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE employment_contract
    SET
        employee_category = p_new_category,
        salary = salary * 1.3
    WHERE id_employee = p_employee_id;
END;
$$;

```

Триггеры:

1. Автоматическое создание статуса заказа при создании заказа.

```

CREATE OR REPLACE FUNCTION create_order_status()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO order_status (
        id_order,

```

```

        change_time,
        status_name
    )
    VALUES (
        NEW.id_order,
        NEW.order_time,
        'принят'
    );

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

INSERT INTO orders (id_employee_shift, order_code, order_time, id_table_shift,
order_cost)
VALUES (261, 'TESTORDER1', TIMESTAMP '2024-01-01 12:00', 1, 0);

db_restaurant=# SELECT id_order_status FROM order_status JOIN orders o ON o.id_order = order_
status.id_order WHERE o.order_code = 'TESTORDER1';
 id_order_status
-----
                17
(1 строка)

```

2. Обновление статуса поставки “не принято”, на который статус всех продуктов из поставки также становится “не принято”.

```

CREATE OR REPLACE FUNCTION sync_supply_status()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.supply_status = 'не принято'
        AND (OLD.supply_status IS DISTINCT FROM NEW.supply_status) THEN

        UPDATE supply_product
        SET status = 'не принято'
        WHERE id_supply = NEW.id_supply;

    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

db_restaurant=# SELECT status FROM supply_product WHERE id_supply = 50 LIMIT 5;
 status
-----
 принято
 принято
 принято
 принято
 принято
(5 строк)

db_restaurant=# UPDATE supply SET supply_status = 'не принято' WHERE id_supply = 50;
UPDATE 1
db_restaurant=# SELECT status FROM supply_product WHERE id_supply = 50 LIMIT 5;
 status
-----
 не принято
 не принято
 не принято
 не принято
 не принято
(5 строк)

```

3. При вставке проверяет, что текущий запас продукта выше минимального.

```

CREATE OR REPLACE FUNCTION check_min_stock_before_order()
RETURNS TRIGGER AS $$
DECLARE
    rec RECORD;
    new_stock NUMERIC;
BEGIN
    FOR rec IN
        SELECT id_product, ingredient_value
        FROM ingredient_dish
        WHERE id_dish = NEW.id_dish
    LOOP
        SELECT current_stock - (rec.ingredient_value / 1000 * NEW.dish_number)
        INTO new_stock
        FROM product
        WHERE id_product = rec.id_product;

        IF new_stock < (SELECT min_stock FROM product WHERE id_product =
rec.id_product) THEN
            RETURN NULL;
        END IF;
    END LOOP;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

db_restaurant=# INSERT INTO dish_order (id_order, id_dish, id_employee, dish_number)
db_restaurant=# VALUES (1, 1, 1, 2000);
INSERT 0 0

```

4. При вставке блюда в заказ вычитает из склада объём ингредиента.

```
CREATE OR REPLACE FUNCTION subtract_ingredients_from_stock()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE product p
    SET current_stock = p.current_stock - (
        idg.ingredient_value / 1000 * NEW.dish_number
    )
    FROM ingredient_dish idg
    WHERE idg.id_dish = NEW.id_dish AND p.id_product = idg.id_product;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

db_restaurant=# SELECT current_stock FROM product WHERE id_product = 2;
               current_stock
-----
 54.550000000000000000000000
(1 строка)

db_restaurant=# INSERT INTO dish_order (id_order, id_dish, id_employee, dish_number)
db_restaurant=# VALUES (1, 1, 1, 1);
INSERT 0 1
db_restaurant=# SELECT current_stock FROM product WHERE id_product = 2;
               current_stock
-----
 54.400000000000000000000000
(1 строка)
```

5. При вставке продукта из поставки увеличивает объём продукта на складе.

```
CREATE OR REPLACE FUNCTION add_stock_from_supply()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.status = 'принято' THEN
        UPDATE product
        SET current_stock = current_stock + NEW.quantity
        WHERE id_product = NEW.id_product;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```

db_restaurant=# SELECT current_stock FROM product WHERE id_product = 2;
      current_stock
-----
 54.400000000000000000000000
(1 строка)

db_restaurant=# INSERT INTO supply_product (id_supply, id_product, quantity, status, origin_c
country,
db_restaurant=# calorie_content, producter, price, production_time, expiration_time)
db_restaurant=# VALUES (12, 2, 20, 'принято', 'Россия', 200, '000', MONEY(100), TIMESTAMP '20
24-01-01 00:00', TIMESTAMP '2024-12-31 00:00');
INSERT 0 1
db_restaurant=# SELECT current_stock FROM product WHERE id_product = 2;
      current_stock
-----
 74.400000000000000000000000
(1 строка)

```

6. Обновление стоимости поставки при добавлении в неё продуктов.

```

CREATE OR REPLACE FUNCTION update_supply_supply_price()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE supply s
    SET supply_price = (
        SELECT SUM(sp.price * sp.quantity)
        FROM supply_product sp
        WHERE sp.id_supply = s.id_supply
    )
    WHERE s.id_supply = NEW.id_supply;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

db_restaurant=# SELECT supply_price FROM supply WHERE id_supply = 12;
      supply_price
-----
 105 886,00 ?
(1 строка)

db_restaurant=# INSERT INTO supply_product (id_supply, id_product, quantity, status, origin_c
country, calorie_content, producter, price, production_time, expiration_time)
db_restaurant=# VALUES (12, 2, 10, 'принято', 'Россия', 100, '000', MONEY(50), TIMESTAMP '202
4-01-01 00:00', TIMESTAMP '2024-12-31 00:00');
INSERT 0 1
db_restaurant=# SELECT supply_price FROM supply WHERE id_supply = 12;
      supply_price
-----
 106 386,00 ?
(1 строка)

```

7. Обновление стоимости заказа при добавлении в него блюда

```

CREATE OR REPLACE FUNCTION update_order_cost()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE orders o
    SET order_cost = (
        SELECT SUM(dp.dish_price * d.dish_number)
        FROM dish_order d
        JOIN dish_price dp

```



```

        ON dp.id_dish = d.id_dish
    WHERE d.id_order = NEW.id_order
)
WHERE o.id_order = NEW.id_order;

RETURN NEW;
END;
$$ LANGUAGE plpgsql;

db_restaurant=# SELECT order_cost FROM orders WHERE id_order = 1;
 order_cost
-----
 3 100,00 ?
(1 строка)

db_restaurant=# INSERT INTO dish_order (id_order, id_dish, id_employee, dish_number)
db_restaurant=# VALUES (1, 1, 1, 1);
INSERT 0 1
db_restaurant=# SELECT order_cost FROM orders WHERE id_order = 1;
 order_cost
-----
 3 420,00 ?
(1 строка)

```

Доп. задание

Триггер учитывает ситуации, когда первым вставляется выход из здания, вход и выход в одно время, вставка входа/выхода между валидными записями, вставка записей в будущее.

```

CREATE OR REPLACE FUNCTION fn_check_time_punch()
RETURNS TRIGGER AS $psql$
DECLARE
    v_prev BOOLEAN;
    v_next BOOLEAN;
    v_same_time BOOLEAN;
BEGIN
    SELECT EXISTS (
        SELECT 1
        FROM time_punch tp
        WHERE tp.employee_id = NEW.employee_id
            AND tp.punch_time = NEW.punch_time
    )
    INTO v_same_time;

    IF v_same_time THEN
        RETURN NULL;
    END IF;

    SELECT tp.is_out_punch
    INTO v_prev
    FROM time_punch tp

```

```

WHERE tp.employee_id = NEW.employee_id
      AND tp.punch_time < NEW.punch_time
ORDER BY tp.punch_time DESC, tp.id DESC
LIMIT 1;

SELECT tp.is_out_punch
INTO v_next
FROM time_punch tp
WHERE tp.employee_id = NEW.employee_id
      AND tp.punch_time > NEW.punch_time
ORDER BY tp.punch_time ASC, tp.id ASC
LIMIT 1;

IF v_prev IS NULL THEN
    IF NEW.is_out_punch = TRUE THEN
        RETURN NULL;
    END IF;
ELSE
    IF v_prev = NEW.is_out_punch THEN
        RETURN NULL;
    END IF;
END IF;

IF v_next IS NOT NULL THEN
    IF v_next = NEW.is_out_punch THEN
        RETURN NULL;
    END IF;
END IF;

IF NEW.punch_time > CURRENT_TIMESTAMP THEN
    RETURN NULL;
END IF;

RETURN NEW;
END;
$psql$ LANGUAGE plpgsql;

```